

- FK can have null value

- I could modify my existing implementation of Project 1.)

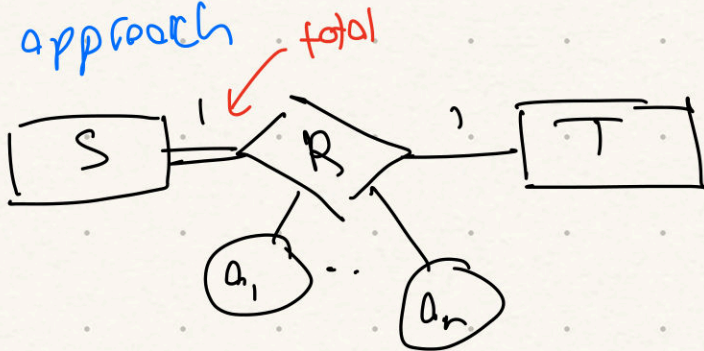
Mapping Binary 1:1 Relationship Sets

- F.K. approach

- Merged relation approach

(if both participations are total, merge 2 entity sets and relationship set into single table)

① F.K. approach



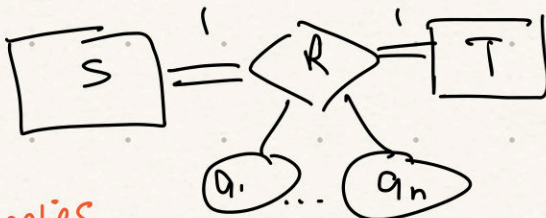
Cross Ref also applies here

merge-rel can work here with F.K. from total participation - not desirable

S	<u>PK(S)</u>	...	PK(T)	a ₁	...	a _n
---	--------------	-----	-------	----------------	-----	----------------

T	<u>PK(T)</u>	...	
---	--------------	-----	--

② merge-relation



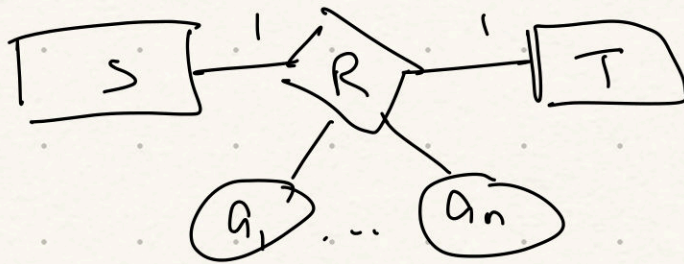
P.K. can be either one

FK applies to second case as well
Cross Ref can also apply here

ST	<u>PK(S)</u>	PK(T)	...	a ₁	...	a _n
----	--------------	-------	-----	----------------	-----	----------------

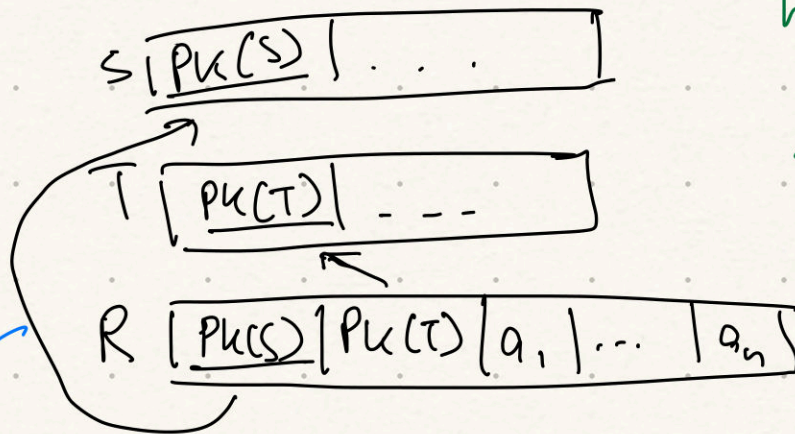
(since it's 1:1)

② When none is total (Cross-reference)



FK approach can
also apply here

can't have
merge-relation
here, could have
null values
from either S or T

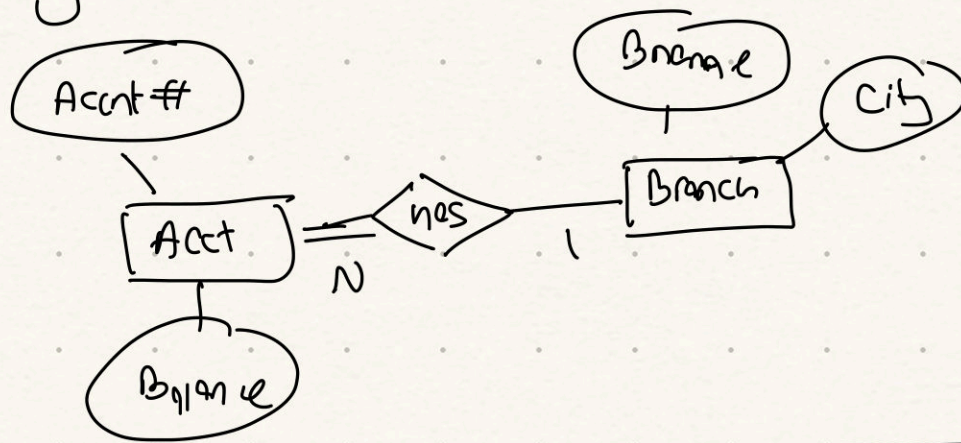


either
one can
be unique.

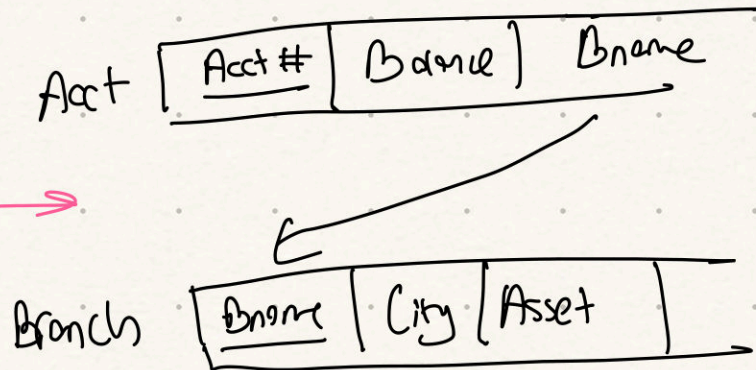
if combo is unique,
either is unique

Since 1:1

* Mapping of Binary 1:N Relationship Sets

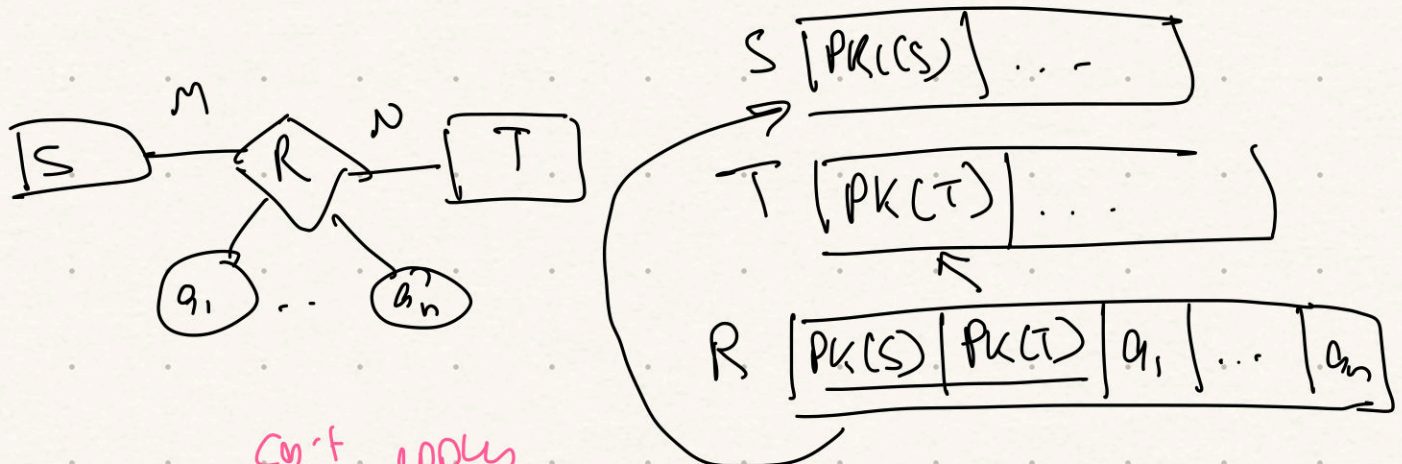


since
N:1



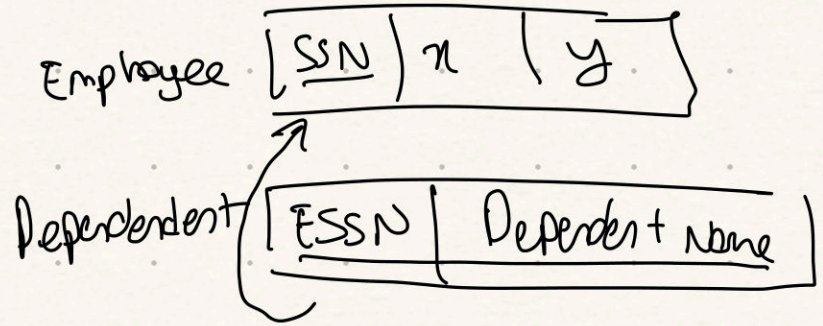
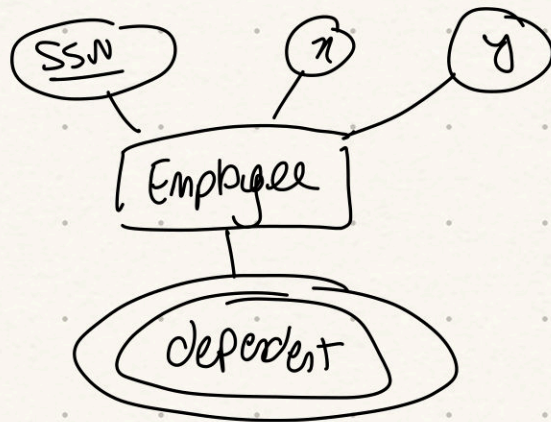
* Mapping Binary M:N relationship Sets

- always create separate table for relationship set if M:N
- relationship attr can't be shifted



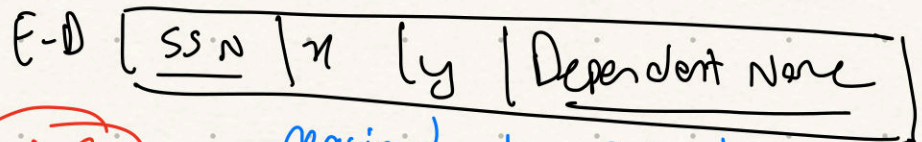
can't apply
F-K approach here since
it could create circular dependency

* Mapping of Multivalued Attributes



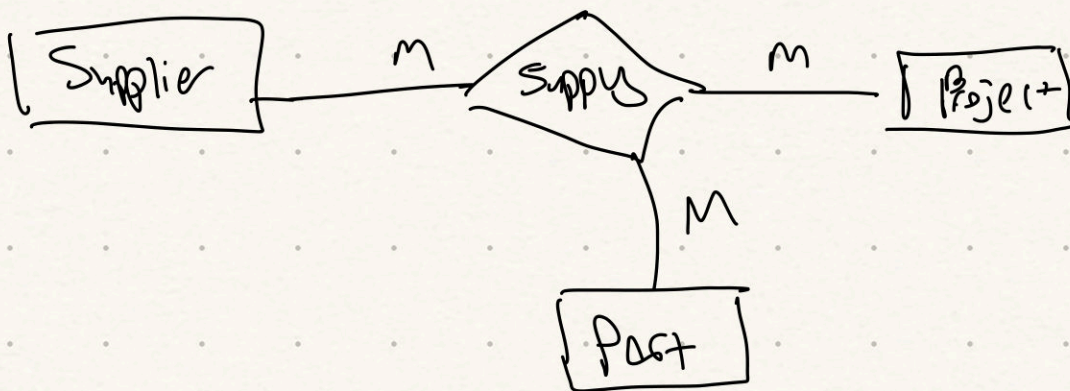
Combination
to form
P.K

alternatively,



required to repeat
n and y for each
dependent, not ideal

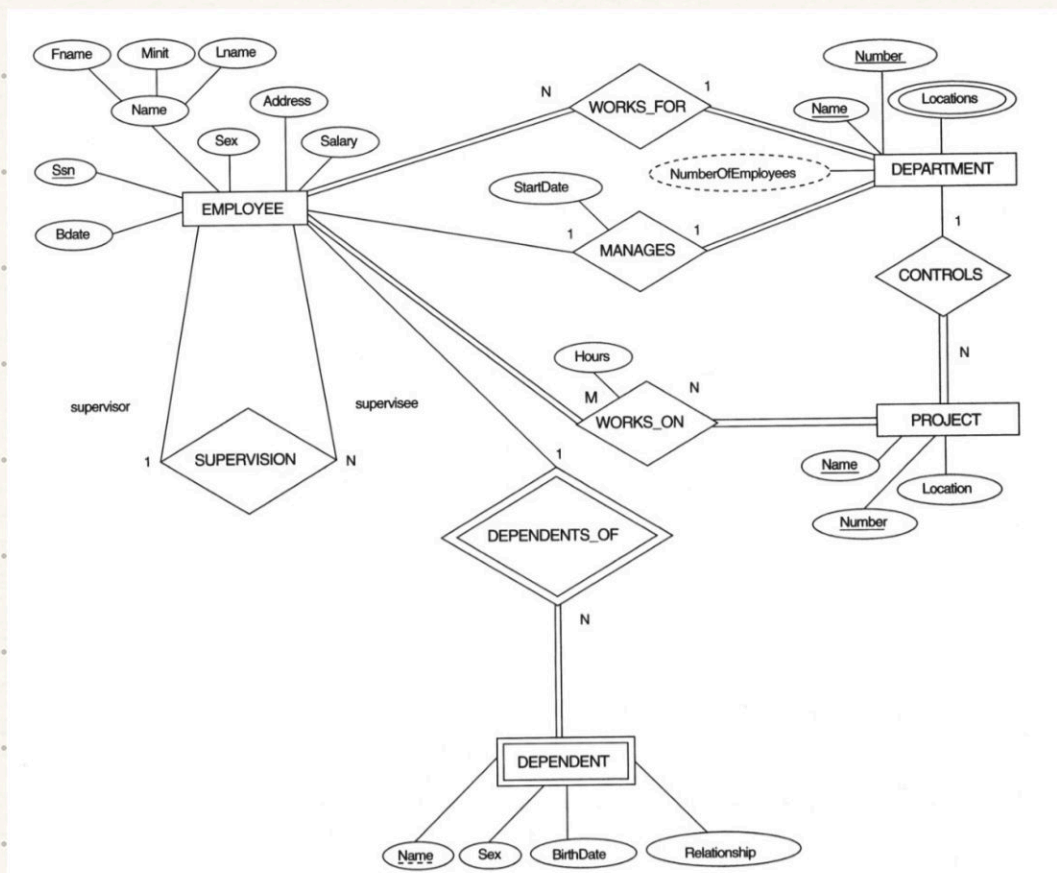
* Mapping of N-ary relationship sets

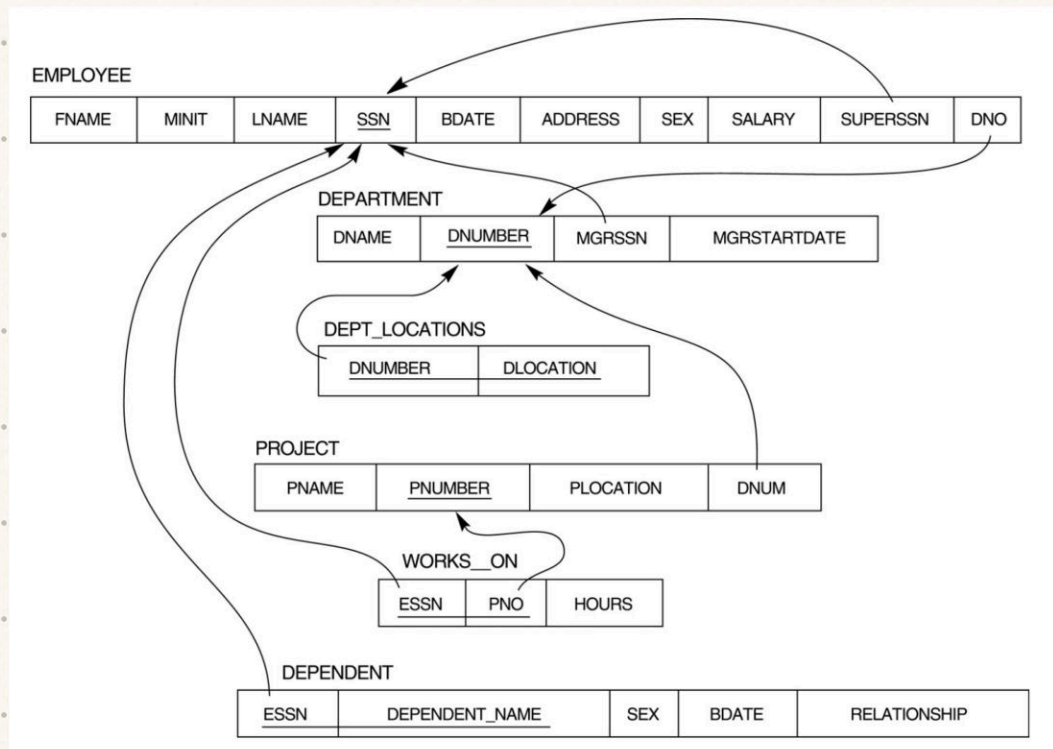


- Mapping Aggregation

- must have tables for each strong entity sets

- primarily creating the relationship set to a separate table





Customer

<u>SSN</u>	Name
------------	------

Loan

<u>Loan #</u>	Amount
---------------	--------

Borrower

<u>PK(C)</u>	<u>PK(L)</u>
--------------	--------------

Employee

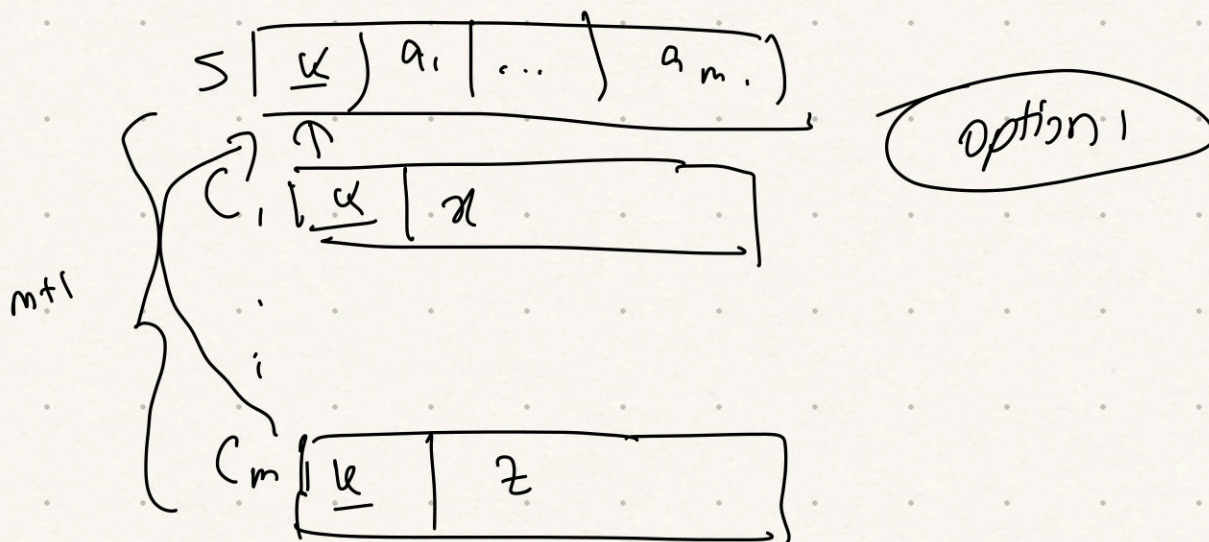
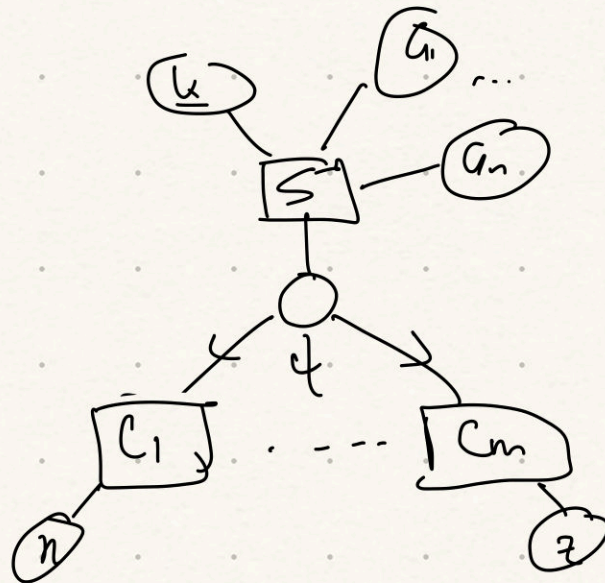
<u>ESSN</u>	Name	Addr
-------------	------	------

Loan Officer

<u>PK(E)</u>	<u>PK(B)</u>
--------------	--------------

Full participation between borrower and loan officer, then can remove borrower for redundancy

Mapping of Specialization / Generalization



Redundancy with option 2

Options: works only with disjoint
- single relation with one type attribute

Review these slides . . .
last concentration . . .

if total participation and disjoint,
all options apply . . .

Vehicle (VIN, Price, Model, Engine size ..., Type)

Sales Person (Sid, Name)

Customer (SSN, Name, Street, City, State)

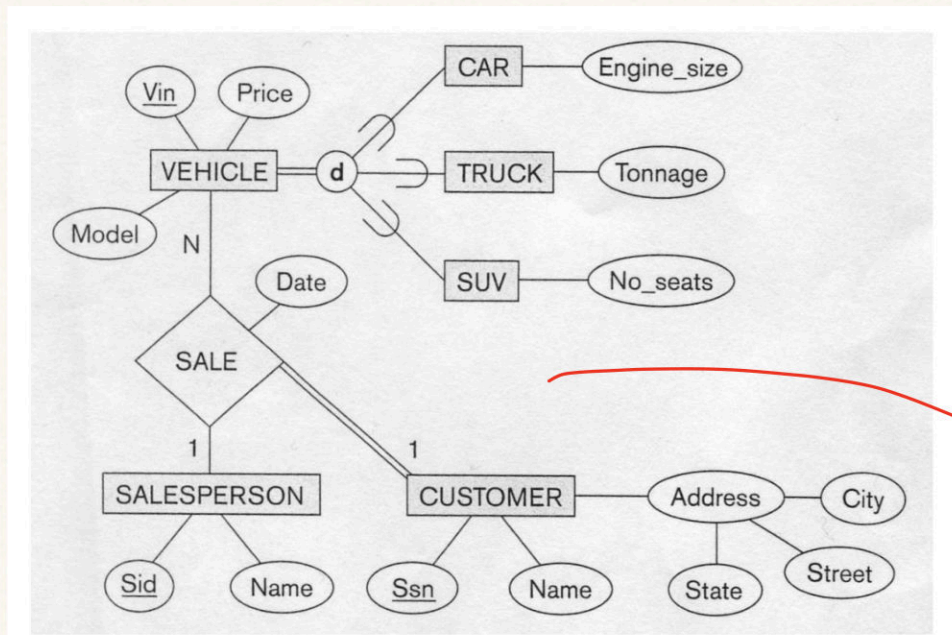
Sale (VIN, Sid, SSN)

syn about
option 2?

→ P.K. reduced to minimal set
since its not a ternary relationship

alternatively using option 2,

Car (VIN, Price, Model, Engine size)



we can convert it
to a binary relation. we can merge Vehicle
and Sale since they share the same P.K.

Lecture 6

Relational Algebra

- σ used to select tuples
- selecting tuples from relation R that satisfy selection condition C

$$\sigma_C(R)$$

String constant written in single quotes

e.g.

$$\sigma_{(Dno=4 \text{ AND } Salary > 25000) \text{ OR } (Dno=5 \text{ AND } Salary > 30000)}$$

(EMPLOYEE)

- NOT > AND > OR - Precedence

- σ is commutative

- Project used to SELECT attributes

$$\pi_L(R)$$

↖ list of attributes

$$\pi_{Rid, Height}(Routes)$$

↳ basically filtering by columns

- in 'pure' relational algebra, selections should be sets, so no duplicates

- But in SQL, it's a multiset

- π is unary, not commutative

$\pi_{name, age} (\sigma_{age > 30} (Climbers))$

↙ SELECT name, age from CLIMBERS
WHERE age > 30

- ideally we apply the filter first and then project attributes

Do we need to check union compatibility for Cartesian product?

- No, since it already returns all list of combination uniquely

Customer: C_Name, C_Street, C_City

Depositor: C_Name, Account#

Loan: B_Name, Loan#, Amount

Borrower: C_Name, Loan#

Select tuples of the loan relation where the branch name is "GMU"

$\sigma_{branch = 'GMU'} (Loan)$

- Find all tuples in which the amount of loan is more than \$12000.

$\sigma_{amount > 12000} (Loan)$

- Find those tuples pertaining to loans of more than \$12000 made by "GMU" branch.

$\sigma_{amount > 12000 \text{ AND } branch = 'GMU'} (Loan)$

- List all loan numbers and the amount of loan.

$\pi_{loan\#, amount} (Loan)$

- Find the names of all customers who live in "airfax".

$\pi_{name} (\sigma_{city = 'airfax'} (Customer))$

- Find the names of all bank customers who have either an account or a loan or both.

$\pi_{name} (Depositor) \cup \pi_{name} (Borrower)$

- Find the names of all customers of the bank who have an account but not a loan.

$\pi_{name} (Depositor) - \pi_{name} (Borrower)$

- Find the names of all customers who have both a loan and an account.

$\pi_{name} (Depositor) \cap \pi_{name} (Borrower)$

Customer: C_Name, C_Street, C_City

Depositor: C_Name, Account#

Loan: B_Name, Loan_#, Amount

Borrower: C_Name, Loan#

- Select tuples of the loan relation where the branch name is "GMU".

$$\sigma_{B_name = 'GMU'} (Loan)$$

- Find all tuples in which the amount of loan is more than \$12000.

$$\sigma_{Amount > 12000} (Loan)$$

- Find those tuples pertaining to loans of more than \$12000 made by "GMU" branch.

$$\sigma_{Amount > 12000 \text{ AND } B_name = 'GMU'} (Loan) \quad \text{or,} \quad R1 \cap R2$$

- List all loan numbers and the amount of loan.

$$\pi_{Loan\#, Amount} Loan$$

- Find the names of all customers who live in "Fairfax".

$$\pi_{name} (\sigma_{C_city = 'Fairfax'} (Customer))$$

- Find the names of all bank customers who have either an account or a loan or both.

$$\pi_{name} (Depositor) \cup \pi_{name} (Borrower)$$

- Find the names of all customers of the bank who have an account but not a loan.

$$\pi_{name} (Depositor) - \pi_{name} (Borrower)$$

- Find the names of all customers who have both a loan and an account.

$$\pi_{name} (Depositor) \cap \pi_{name} (Borrower)$$

- Cartesian Product

$$\sigma_{c1d.1 = c1d.2} (Climbs \times Climbers)$$

In general, very expensive...

Use JOIN instead

- (THETA) JOIN

- Join of two relations

$$R \bowtie_c S = \sigma_c (R \times S)$$

$\hookrightarrow c$ is called a

- usually using the
full constraint.

Join condition

$\hookrightarrow$ can only
take attribute
name

this is different
from SELECTION
condition

$$Climbs \bowtie_{(c1d.1 = c1d.2)} Climbers$$

- there should be fewer tuples in JOIN
compared to PRODUCT

- Natural JOIN

- where the condition is implied

- not knowledge to user

① Sailors $\times$ Reserves (Cartesian, 6 tuples)

② Sailors $\bowtie$ Reserves (Theta Join)
Sailor.Sid < Reserve.sid

S.sid	...	R.sid	...
22		58	
31		58	

③ Sailors $\bowtie_{S.sid = R.sid}$ Reserves (Equi-join)

S.sid	...	R.sid	...
22		22	
58		58	

(7 columns)

④ Sailors $\bowtie$ Reserves (Natural Join) (6 columns)

Sid	...
22	

$\pi_{sname} (\sigma_{r.bid = 103} (Sailors \bowtie_{s.sid = r.sid} Reserves))$

11:10 AM Mon Mar 16 (VPS) 55%

Exercise

Sailors

sid	sname	rating	age
22	Dustin	7	45
31	Lubber	8	55
58	Rusty	10	35

Boats

bid	bname	color
101	Interlake	blue
102	Interlake	red
103	Clipper	green
104	Marine	red

Reserves

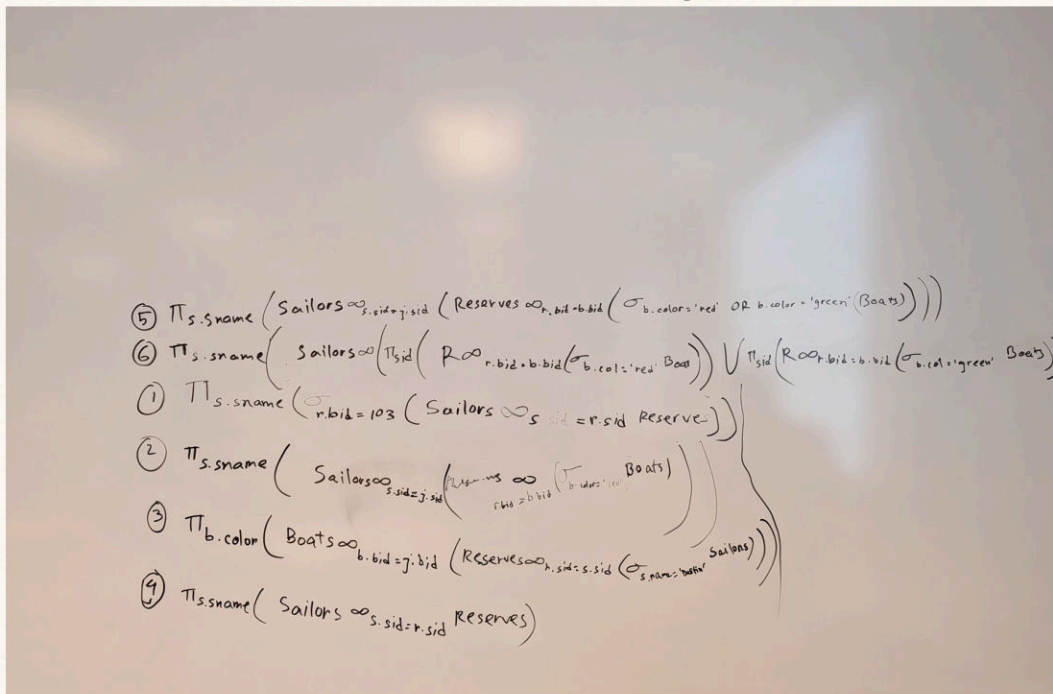
sid	bid	day
22	101	10/10/96
58	103	11/12/96

Exercise (Cont.)

- Find the names of sailors who have reserved boat 103.
- Find the names of sailors who have reserved a red boat.
- Find the colors of boats reserved by Dustin.
- Find the names of sailors who have reserved at least one boat.
- Find the names of sailors who have reserved a red or a green boat.
- Find the names of sailors who have reserved a red and a green boat.
- Find the names of sailors with age over 20 who have

- ① $\pi_{sname} (Sailors \bowtie_{b.bid = 103} Reserves)$
- ② $\pi_{sname} (\sigma_{color = 'red'} (Sailors \bowtie_{s.sid = r.sid} (Boats \bowtie Reserves)))$
- ③ $\pi_{color} (\sigma_{sname} (Sailors) \bowtie Reserves \bowtie Boats)$
- ④ $\pi_{sname} (Sailors \bowtie Reserves)$

⑤ $\pi_{sname} ((\sigma_{color='red'} \text{ OR } color='green' \text{ Boats}) \bowtie R \bowtie B)$ ↖ ??



⑥ Red $\rightarrow \pi_{sb, name} (S \bowtie R \bowtie (\sigma_{color='red'} B)) \bowtie S$

Green $\rightarrow \pi_{sb, name} (S \bowtie R \bowtie (\sigma_{color='green'} B)) \bowtie S$

$R \cap G \leftarrow \pi_{sname} (Red \cap Green)$

$$\begin{aligned}
 R_1 &\leftarrow \pi_{sid} (\sigma_{age > 20} \text{ Sailors}) \\
 R_2 &\leftarrow \pi_{sid} (R \bowtie (\sigma_{color = 'red'} B)) \\
 R &\leftarrow \pi_{sname} ((R_1 - R_2) \bowtie \text{Sailors})
 \end{aligned}$$

- Division in SQL query

Climbs		C x R	
Cid	sid	Cid	Rid
1	A	1	A
1	B	1	B
2	A	2	A
		2	B

$$C \times R - \pi_{cid, rid} (Climbs)$$

or,

$$\pi_{cid, rid} (Climbs) \div (\pi_{rid} Routes)$$

Climbers who have climbed all routes

What about "Routes that have been climbed by all climbers"?

Confused on division...

- division keyword: ALL

① find rows of sailors who have reserved
all boats
→ denominator

$$R1 \leftarrow \pi_{sid, bid} (reserves) / \pi_{sid} (Boats)$$

$$R \leftarrow \pi_{sname} (R1 \bowtie Sailors)$$

② $R_1 \leftarrow (\pi_{sid, bid} reserves) \div \pi_{bid} (\delta_{name = '...'} Boats)$

Recap of Relational Algebra Operations

• Select	$\sigma < \text{selection condition} > R$
• Project	$\pi < \text{attribute list} > R$
• Union	$R \cup S$
• Intersection	$R \cap S$
• Difference	$R - S$
• Cross product	$R \times S$
• Join	$R \bowtie < \text{join condition} > S$
• Natural join	$R \natural \bowtie S$
• Division	$R \div S$
• Rename	$\rho < \text{new schema} > R$

Sailors (sid, sname, rating, age)

Boats (bid, bname, color)

Reserves (sid, bid, day)

- Find the names of sailors who have reserved all boats.

$$\rho(R_1, \pi_{\text{bid}} \text{Reserves} \div \pi_{\text{bid}} \text{Boats}) \\ \pi(R_1 \bowtie \text{Sailors})$$

- Find the names of sailors who have reserved all boats called Interlake.

- Find the names of sailors who have reserved at least two different boats.

$$R_1 \bowtie \text{Reserves}$$

$$R_1.\text{sid} = \text{Reserves}.\text{sid} \text{ AND}$$

$$R_1.\text{bid} \neq R_2.\text{bid}$$

- Find the names of sailors who have reserved the same boat at least twice.

$$\rho R_1 \text{Reserves}$$

$$T_1 \leftarrow \pi_{\text{sid}} (R_1 \bowtie \text{Reserves})$$

$$R_1.\text{sid} = \text{Reserves}.\text{sid} \text{ AND}$$

$$R_1.\text{bid} = \text{Reserves}.\text{bid} \text{ AND}$$

$$R_1.\text{day} \neq \text{Reserves}.\text{day}$$

$$R \leftarrow \pi_{\text{name}} (T_1 \bowtie \text{Sailors})$$

$$\rho(R_2 \text{Reserve})$$

- essentially creating multiple renames / aliases as needed

- PREP FOR QUIZ

Project 1-3